

SP-4 Phase E — Credentials sync via Lava API (detailed design)

Spec date: 2026-05-13 **Parent SP:** docs/superpowers/specs/2026-05-12-sp4-multi-provider-redesign.md **Prerequisite:** Phase G.1 complete (HEAD 68f75e6a) — soft-delete + outbox enqueue local half landed. **Status:** detailed-design locked. Execution requires the Lava Go API service (lava-api-go/) to gain three new endpoints + operator-driven end-to-end integration testing. Implementation plan TBD per a separate Go-side cycle.

Goal (from parent SP-4 design)

Phase E — Credentials sync via Lava API. New Go endpoints `/v1/credentials` (PUT/DELETE/GET) with per-user key encryption.

Why Phase E waits

Phase G.1 ships the local half of removal-sync: the soft-delete + outbox enqueue. The outbox now accumulates CREDENTIALS entries (upserts AND removals via `WireRemoval { id, deletedAt, deleted: true }`). Without Phase E:

- Sync-down does not exist — a fresh device install or a second device never sees the user's credentials.
- Sync-up does not exist — local changes never reach the API.
- Backup-restore reanimation IS already prevented (the soft-delete marker is in the Room backup; the restored device respects `WHERE deletedAt IS NULL`).

Phase E closes the cross-device + fresh-install gaps.

Cryptographic envelope (locked in parent SP-4 design)

AES-256-GCM with a per-user passphrase. Key derived via PBKDF2-SHA-256 (200_000 iterations). Zero-knowledge: API stores only encrypted blobs.

Phase A's `CredentialsCrypto` already does this for at-rest encryption. Phase E uses a SEPARATE per-user key for transport:

1. **At-rest key** (Phase A): derived from passphrase + `saltA`. Used by `CredentialsCrypto.encrypt/decrypt` for the ciphertext column of `credentials_entry`.

2. **Transport key** (Phase E, NEW): derived from passphrase + saltB (saltB is a separate constant). Used to encrypt the wire payload uploaded to /v1/credentials.

Two-salt construction so the at-rest key NEVER leaves the device. The transport key is also derived locally; the Go API sees only ciphertext.

saltB is stored client-side. The Go API never receives or generates it.

Go API endpoints (NEW)

PUT /v1/credentials/:id

Request:

```
{
  "displayName": "...",
  "type": "USERNAME_PASSWORD|API_KEY|BEARER_TOKEN|COOKIE_SESSION",
  "ciphertext": "<base64 of AES-GCM transport-encrypted body>",
  "iv": "<base64 of 12-byte IV>",
  "createdAt": 1700000000000,
  "updatedAt": 1700000000000,
  "deletedAt": null
}
```

Server-side:

- Authorize by Lava-Auth (existing per-build chain).
- Store the row keyed by (user_id, id).
- (user_id, id) is the primary key — re-PUTs replace. Last-Write-Wins by updatedAt.
- The server has NO access to plaintext. ciphertext is opaque.

DELETE /v1/credentials/:id

Idempotent. Marks the row deletedAt = NOW() (soft-delete on the server too, to support cross-device propagation to other clients). Returns 204.

GET /v1/credentials?since=<utc-ms>

Lists rows for the authenticated user with updatedAt > since (delta sync). Includes soft-deleted rows so the client can apply local soft-deletes.

Client-side sync mechanism

Upload (drains the outbox)

A `CredentialsSyncWorker` (`WorkManager`) runs:

1. Reads the next CREDENTIALS outbox entry.
2. If payload has "deleted":true → DELETE /v1/credentials/:id.
3. Else (upsert) → PUT /v1/credentials/:id with the transport-encrypted body.
4. On 2xx, ack(outboxId) to remove the outbox row.
5. On 4xx/5xx, exponential backoff retry.

Download (delta sync)

On app foreground OR periodic (15 min):

1. Read lastSyncedAt from preferences.
2. GET /v1/credentials?since=lastSyncedAt.
3. For each remote row:
 - If deletedAt != null AND local row exists → dao.softDelete(id, deletedAt).
 - Else if local row missing OR remote updatedAt > local.updatedAt → decrypt + dao.upsert(...).
4. Store new lastSyncedAt.

Conflict resolution

Last-Write-Wins by updatedAt. A simultaneous edit on two devices is rare (credentials change infrequently); LWW is adequate.

Affected surfaces (placeholder — full plan filed in a separate cycle)

- lava-api-go/cmd/server/: new credentials_handler.go + tests.
- lava-api-go/internal/storage/: new schema for user_credentials table.
- core/sync/: new CredentialsSyncWorker (WorkManager).
- core/credentials/: new transport key derivation alongside the at-rest key.
- core/network/impl/: HTTP client surface for the three new endpoints.

Anti-bluff posture

- API tests under lava-api-go/tests/contract/ MUST hit a real Postgres in a container per §6.K + the existing Go test pattern.
- Client tests MUST drive the real CredentialsSyncWorker against a real OkHttp + a MockWebServer simulating the API.

- Falsifiability rehearsal per test: revert the per-user-key derivation, observe that the API rejects the wrong-cipher payload OR the decrypt round-trip fails.

Operator-driven execution gates

Phase E execution requires:

1. Operator-supplied lava-api-go local instance with the new endpoints implemented.
2. Operator-supplied test user account on the API.
3. Backup-restore Challenge Test on a second emulator (per §6.I matrix).

Why design only (no implementation in this session)

Phase E is multi-domain (Go + Android), needs a working API instance, requires user-account provisioning, and needs cross-device integration testing on a second emulator. None of those are reachable from an autonomous Android-side session. The design exists so the next operator-driven cycle has a contract to execute against.